

Lecture Scribes - Lecture Scribe 10



L10_S1.pdf

PDF

Generate a lecture scribe based on the attached materials that is intended to serve as reference material for exam preparation. While writing the scribe, you must reason step-by-step through all definitions, assumptions, proofs, and example solutions exactly as they are presented in the provided context.

Ensure the logical flow is made explicit and all reasoning is included in the final scribe so a student can follow it during revision. Do not invent content, introduce new examples, or include information not found in the lecture slides or textbook. Maintain a formal academic tone and follow the structure and scope of the lecture.

Below is a **formal lecture scribe for exam preparation** reconstructed strictly from the uploaded lecture slides. The explanations follow the **same structure and scope as the lecture** and include **step-by-step reasoning for definitions, assumptions, algorithms, and pseudocode**, without introducing any external information.

Source:

Lecture Scribe

CSE400 – Fundamentals of Probability in Computing

Lecture 10: Randomized Min-Cut Algorithm

1. Introduction to the Min-Cut Problem

1.1 Motivation: Why Use Min-Cut?

The min-cut algorithm is used in several real-world scenarios where **network connectivity, reliability, and optimization** must be analyzed. The core idea is to determine the **smallest**

set of edges whose removal disconnects a graph.

The slides identify several important applications.

Network Design

The min-cut algorithm helps improve communication efficiency and optimize network flow.

Step-by-step reasoning:

1. A network can be represented as a **graph**.
2. Nodes represent entities (computers, routers, etc.).
3. Edges represent communication links.
4. Removing certain links may break connectivity.
5. The **minimum cut identifies the smallest number of links whose removal disconnects the network**.

This information helps engineers determine **weak points in the network** and design stronger communication systems.

Communication Networks

In communication systems, networks must remain reliable even when failures occur.

Reasoning process:

1. Failures can occur due to damaged links or overloaded connections.
2. If a small number of edges can disconnect the network, the system is vulnerable.
3. By computing the **minimum cut**, engineers identify the most critical edges.
4. Protecting these edges increases **fault tolerance**.

Thus, the min-cut algorithm helps design **robust communication networks**.

VLSI Design

In Very Large Scale Integration (VLSI), circuits contain millions of components connected by wires.

Reasoning:

1. Circuits can be represented as graphs.
2. Nodes represent circuit components.

3. Edges represent connections between components.
4. Partitioning the circuit into smaller components reduces complexity.

The min-cut algorithm helps divide circuits into parts while minimizing the number of connections between them.

2. Definition of the Min-Cut Problem

2.1 Cut-Set

A **cut-set** is defined as:

A set of edges whose removal breaks the graph into two or more connected components.

Logical reasoning behind this definition

1. A graph consists of vertices and edges.
2. Vertices represent objects.
3. Edges represent relationships between them.
4. If certain edges are removed, the graph may become disconnected.
5. The edges whose removal causes this disconnection form a **cut-set**.

Thus, a cut-set represents **edges responsible for maintaining connectivity**.

2.2 Minimum Cut

Given a graph

$$G = (V, E)$$

where:

- V = set of vertices
- E = set of edges
- n = number of vertices

The **minimum cut problem** is defined as:

Finding a cut-set with the smallest possible number of edges.

Step-by-step reasoning

1. Multiple cut-sets may exist in a graph.

2. Each cut-set has a certain size (number of edges).
3. The smallest such cut-set represents the **minimum cut**.

Thus,

$$\text{Min Cut} = \min(\text{size of all possible cut-sets})$$

3. Key Operation: Edge Contraction

The fundamental operation used in randomized min-cut algorithms is **edge contraction**.

Definition

Edge contraction is an operation that:

1. Removes an edge from the graph.
2. Merges the two vertices connected by that edge into a single vertex.

This process is illustrated in the diagram on **page 15 of the slides**, where two vertices connected by an edge become one merged vertex after contraction.

Detailed Contraction Process

When contracting an edge (u, v) :

Step-by-step:

1. Select edge (u, v) .
2. Merge vertices u and v into a new single vertex.
3. Remove the edge connecting u and v .
4. Preserve all other edges connected to either vertex.
5. If duplicate edges appear, they become **parallel edges**.
6. Self-loops are removed.

Therefore the resulting graph:

- May contain **parallel edges**
 - Does **not** contain self-loops.
-

4. Successful Min-Cut Run

A **successful min-cut run** occurs when the algorithm correctly identifies the minimum cut of a graph.

Reasoning

1. The algorithm repeatedly performs edge contractions.
2. During contractions, some edges may belong to the minimum cut.
3. If these edges are **not contracted**, the algorithm preserves the minimum cut structure.
4. Eventually the graph is reduced to two vertices.
5. The edges between those vertices represent the minimum cut.

The figure on **page 19** visually demonstrates a sequence of contractions leading to a correct minimum cut.

5. Unsuccessful Min-Cut Run

An **unsuccessful run** occurs when the algorithm fails to identify the correct minimum cut.

Reasoning

1. The algorithm randomly selects edges for contraction.
2. If an edge belonging to the true minimum cut is contracted early,
3. the structure of the minimum cut is destroyed.
4. The final cut produced will therefore be incorrect.

The diagram on **page 22** shows an example where the algorithm chooses edges that lead to an incorrect cut.

This demonstrates why randomized algorithms must often be **repeated multiple times**.

6. Max-Flow Min-Cut Theorem

The **Max-Flow Min-Cut theorem** establishes a fundamental relationship between flows and cuts in a network.

Theorem Statement

In a flow network, the maximum amount of flow from source to sink equals the total weight of edges in a minimum cut.

6.1 Key Definitions

Capacity of a Cut

The capacity of a cut equals:

Sum of capacities of edges from $X \rightarrow Y$

where the graph is partitioned into two sets X and Y .

Minimum Cut

The cut with the **smallest total capacity** among all possible cuts.

Maximum Flow

The **largest possible flow** that can move from the **source (S)** to the **sink (T)** in the network.

Logical Relationship

The theorem states:

$$\text{Maximum Flow} = \text{Minimum Cut Capacity}$$

The network diagram on **page 26** illustrates this relationship between flow values and the cut separating source and sink.

7. Deterministic Min-Cut Algorithm

A deterministic algorithm always produces the **exact minimum cut**.

The lecture discusses the **Stoer–Wagner Minimum Cut Algorithm**.

7.1 Stoer–Wagner Theorem

Let s and t be vertices in graph G .

Let:

$$G/s, t$$

represent the graph obtained by **merging vertices s and t** .

The theorem states:

A minimum cut of G is the smaller of:

1. Minimum $s - t$ cut in G
2. Minimum cut in $G/s, t$

Logical explanation

Two possibilities exist:

1. The minimum cut separates s and t .
 - Then the **minimum $s - t$ cut** is the answer.
 2. The minimum cut does **not separate s and t** .
 - In this case merging them does not change the minimum cut.
 - Therefore the minimum cut of $G/s, t$ is the answer.
-

8. Deterministic Algorithm Pseudocode

Algorithm 1: MinimumCutPhase(G, a)

Step-by-step logic:

1. Initialize set

$$A = \{a\}$$

2. While $A \neq V$
 - Add the **most tightly connected vertex** to A .
 3. When all vertices are added:
 - Return the cut weight called **cut of the phase**.
-

Algorithm 2: MinimumCut(G)

Step-by-step reasoning:

1. While the number of vertices $|V| \geq 1$
2. Choose any vertex a .
3. Run:

$$\text{MinimumCutPhase}(G, a)$$

4. Compare the phase cut with the current minimum cut.
5. If it is smaller:
 - Update the minimum cut.
6. Shrink the graph by merging the last two vertices added.
7. Repeat until termination.

Finally:

Return the minimum cut

9. Randomized Min-Cut Algorithm

Randomized algorithms use **random decisions** during execution.

9.1 Why Randomized Algorithms?

The slides list several reasons.

1. Probabilistic Guarantee

The algorithm provides a probability of successfully finding the minimum cut.

2. Efficiency

Randomized approaches may be faster for large graphs.

3. Parallelization

Multiple independent runs can execute simultaneously.

4. Approximation

They produce a close estimate of the minimum cut.

5. Avoidance of Worst-Case Instances

Randomness prevents adversarial inputs from forcing worst-case performance.

6. Robustness

The algorithm performs well across many types of graphs.

10. Karger's Randomized Min-Cut Algorithm

Karger's algorithm repeatedly performs **random edge contraction**.

Conceptual Steps

1. Start with graph G .
2. Randomly select an edge.
3. Contract the edge.

4. Repeat until only **two vertices remain**.
5. The edges between these two vertices represent the cut.

The diagram on **page 34** shows an example run where certain random edge choices lead to a non-optimal cut.

11. Recursive Randomized Min-Cut Pseudocode

Algorithm: Recursive-Randomized-Min-Cut(G, α)

Input:

- Undirected multigraph G
- Constant $\alpha > 0$

Output:

- A cut C
-

Step-by-Step Logic

1. If the number of vertices $n \leq \alpha^3$
 - Compute the min cut using **brute force search**.
2. Otherwise:

Repeat for $i = 1$ to α

- a. Create graph G' by performing random contractions.
- b. Recursively compute:

$$C' = \text{RecursiveRandomizedMinCut}(G', \alpha)$$

c. If

$$i = 1 \quad \text{or} \quad |C'| < |C|$$

update

$$C = C'$$

3. Return C .
-

12. Comparison: Deterministic vs Randomized Min-Cut

Deterministic Min-Cut

Characteristics:

- Always produces the **exact minimum cut**.
- May have **higher computational cost**.

Time complexity (Stoer–Wagner):

$$O(VE + V^2 \log V)$$

Randomized Min-Cut

Characteristics:

- Produces **approximate results with high probability**.
- Faster for large graphs.

Time complexity (Karger's algorithm):

$$O(V^2)$$

Thus the choice depends on the **specific problem requirements**.

13. Probability Theorem for Min-Cut

The lecture states the probability guarantee:

$$P(\text{algorithm outputs minimum cut}) \geq \frac{2}{n(n-1)}$$

where n is the number of vertices.

This probability explains why the algorithm must be **repeated several times** to increase success probability.

14. Python Simulation

Students are instructed to:

1. Open the **Campuswire post for Lecture 10**.
 2. Download the provided **Jupyter Notebook (.ipynb)**.
 3. Run the simulation to observe Karger's randomized algorithm in practice.
-

Conclusion

This lecture introduces the **minimum cut problem** and explores both **deterministic and randomized approaches** to solving it.

Key ideas developed step-by-step include:

- Definition of cut-sets and minimum cuts
- Edge contraction as the main operation
- Successful vs unsuccessful randomized runs
- The Max-Flow Min-Cut theorem
- Stoer–Wagner deterministic algorithm
- Karger's randomized min-cut algorithm
- Complexity and probability guarantees

Together, these concepts demonstrate how **randomization can be used effectively to solve graph optimization problems**.

✅ If you want, I can also generate:

- Exam-ready 10 long answers from this lecture
- 2–3 mark short notes for quick revision
- Predicted 40-mark exam paper from this lecture
- 1-page cheat sheet for last-minute study

These will make your exam revision **much faster and easier**.

Generate a lecture scribe based on the attached materials that is intended to serve as reference material for exam preparation. While writing the scribe, you must reason step-by-step through all definitions, assumptions, proofs, and example solutions exactly as they are presented in the provided context.

Ensure the logical flow is made explicit and all reasoning is included in the final scribe so a student can follow it during revision. Do not invent content, introduce new examples, or include information not found in the lecture

slides or textbook. Maintain a formal academic tone and follow the structure and scope of the lecture.

Below is a **formal lecture scribe** constructed strictly from the uploaded lecture slides. The structure, terminology, definitions, and reasoning follow the **exact order and scope presented in the lecture** so it can be used directly for **exam preparation and revision**.

Source material:

Lecture Scribe

CSE400 – Fundamentals of Probability in Computing

Lecture 10: Randomized Min-Cut Algorithm

1. Overview of the Lecture

The lecture focuses on the **minimum cut problem in graphs** and the use of **randomized algorithms** to solve it. The lecture progresses through the following topics:

1. The Min-Cut Problem
2. Why the Min-Cut algorithm is used
3. Definition of Min-Cut
4. Successful Min-Cut Run
5. Unsuccessful Min-Cut Run
6. Max-Flow Min-Cut Theorem
7. Deterministic Min-Cut Algorithm
8. Stoer–Wagner Algorithm
9. Randomized Min-Cut Algorithm
10. Karger's Randomized Algorithm
11. Pseudocode of the algorithm
12. Comparison of deterministic and randomized algorithms
13. Probability theorem for min-cut results
14. Python simulation instructions

These topics collectively explain how **graph partitioning problems** can be solved using **both deterministic and randomized approaches**.

2. The Min-Cut Problem

2.1 Why Use the Min-Cut Algorithm

The min-cut algorithm is used to address problems involving **network connectivity, reliability, and optimization**.

The lecture explains several application domains.

Network Design

The algorithm helps improve communication systems by identifying weak points in a network.

Step-by-step reasoning:

1. Communication networks can be represented as graphs.
2. Nodes represent devices or processing units.
3. Edges represent communication links.
4. Removing certain links may disconnect the network.
5. The goal is to identify the **smallest set of edges whose removal disconnects the network**.

Thus, the min-cut algorithm identifies the **minimum capacity cut** that partitions the network.

Communication Networks

In communication networks, reliability is critical.

Reasoning process:

1. A network may experience failures due to broken links or overloaded channels.
2. Some links are more critical than others.
3. If a small number of edges can disconnect the network, the system is vulnerable.
4. The minimum cut identifies those critical edges.

Therefore, the algorithm helps build **robust and fault-tolerant communication networks**.

VLSI Design

Very Large Scale Integration (VLSI) involves designing circuits with extremely large numbers of components.

Step-by-step reasoning:

1. Circuit components can be represented as vertices.
2. Connections between components are edges.
3. Complex circuits must often be partitioned into smaller subsystems.
4. Partitioning should minimize the number of connections between subsystems.

The min-cut algorithm assists in **partitioning circuits to reduce interconnect complexity**.

3. Definition of Min-Cut

3.1 Cut-Set Definition

A **cut-set** in a graph is defined as:

A set of edges whose removal breaks the graph into two or more connected components.

Logical Interpretation

To understand this definition step-by-step:

1. A graph consists of vertices and edges.
2. The edges provide connectivity between vertices.
3. Removing edges may reduce connectivity.
4. If removing a particular set of edges divides the graph into separate parts, those edges form a **cut-set**.

Thus, a cut-set represents **edges responsible for maintaining connectivity between parts of the graph**.

3.2 Minimum Cut

Given a graph

$$G = (V, E)$$

where:

- V = set of vertices
- E = set of edges
- n = number of vertices

The **minimum cut problem** is defined as:

Finding the cut-set with the minimum cardinality in graph G .

Step-by-Step Reasoning

1. Many cut-sets may exist in a graph.
 2. Each cut-set contains a certain number of edges.
 3. The algorithm evaluates different cut-sets.
 4. The smallest cut-set (with the fewest edges) is called the **minimum cut**.
-

4. Edge Contraction

The lecture states that the **main operation in the min-cut algorithm** is edge contraction.

Definition

Edge contraction is defined as an operation that:

- Removes an edge from a graph
- Merges the two vertices connected by that edge

This concept is illustrated in the diagram on **page 15 of the slides**, where contracting an edge merges two nodes into one.

Detailed Contraction Procedure

When contracting an edge (u, v) :

1. Select the edge connecting vertices u and v .
2. Merge vertices u and v into a single vertex.
3. Remove the edge connecting them.
4. Preserve all other edges that connect to u or v .
5. If multiple edges now connect the same vertices, they become **parallel edges**.
6. Self-loops are removed from the graph.

Thus, the resulting graph may contain **parallel edges but no self-loops**.

5. Successful Min-Cut Run

A **successful min-cut run** refers to a situation where the algorithm correctly finds the minimum cut.

Step-by-Step Reasoning

1. The algorithm repeatedly performs edge contraction.
2. During this process, some edges belong to the true minimum cut.

3. If these edges are **not contracted**, the minimum cut remains intact.
4. Eventually, the graph is reduced to two vertices.
5. The edges connecting those vertices represent the minimum cut.

The diagram on **page 19** demonstrates such a successful sequence of contractions.

6. Unsuccessful Min-Cut Run

An **unsuccessful min-cut run** occurs when the algorithm fails to identify the correct minimum cut.

Step-by-Step Reasoning

1. The algorithm randomly selects edges for contraction.
2. If an edge belonging to the true minimum cut is contracted early,
3. the minimum cut structure is destroyed.
4. As a result, the final cut produced by the algorithm is incorrect.

The diagram on **page 22** illustrates such an unsuccessful run.

7. Max-Flow Min-Cut Theorem

The **Max-Flow Min-Cut Theorem** describes the relationship between network flow and graph cuts.

Theorem Statement

In a flow network, the maximum amount of flow passing from the source to the sink is equal to the total weight of the edges in a minimum cut.

Definitions Used in the Theorem

Capacity of a Cut

The capacity of a cut is defined as:

The sum of the capacities of edges oriented from a vertex in set X to a vertex in set Y .

Minimum Cut

The cut in the network that has the **smallest capacity**.

Minimum Cut Capacity

The total capacity of the edges in the minimum cut.

Maximum Flow

The largest possible flow that can travel from:

- Source S
- to Sink T

in the network.

The network diagram on **page 26** demonstrates this relationship between flow and cut capacity.

8. Deterministic Min-Cut Algorithm

A deterministic algorithm guarantees an exact result.

The lecture introduces the **Stoer–Wagner Minimum Cut Algorithm**.

Stoer–Wagner Theorem

Let:

- s and t be two vertices of graph G .
- $G/s, t$ represent the graph obtained by merging vertices s and t .

The theorem states:

A minimum cut of G is obtained by selecting the smaller value between:

1. The minimum s - t cut in G
 2. The minimum cut of $G/s, t$
-

Logical Justification

Two possibilities exist:

Case 1

The minimum cut separates s and t .

- Then the minimum $s - t$ cut of G is also the minimum cut of the graph.

Case 2

The minimum cut does not separate s and t .

- In this case merging them does not affect the minimum cut.
- Therefore the minimum cut of $G/s, t$ remains valid.

9. Pseudocode for Deterministic Min-Cut

Algorithm 1: MinimumCutPhase(G, a)

Step-by-step procedure:

1. Initialize set

$$A = \{a\}$$

2. While $A \neq V$
 - Add the vertex most tightly connected to set A .
3. Once all vertices are included in A
 - Return the weight of the cut produced in this phase.

Algorithm 2: MinimumCut(G)

1. While the graph contains vertices:
2. Select any vertex a .
3. Execute:

$$\text{MinimumCutPhase}(G, a)$$

4. If the cut obtained in the phase is smaller than the current minimum cut:
 - Store it as the new minimum cut.
5. Shrink the graph by merging the last two vertices added.
6. Continue until all phases are completed.
7. Return the minimum cut.

10. Randomized Min-Cut Algorithm

Randomized algorithms rely on random choices to determine their execution path.

Why Use Randomized Algorithms

The lecture lists several advantages:

1. Probabilistic guarantee of success
 2. Efficiency in large graphs
 3. Parallelization capability
 4. Approximation guarantees
 5. Avoidance of worst-case instances
 6. Robustness across inputs
-

11. Karger's Randomized Algorithm

Karger's algorithm finds the minimum cut through **random edge contraction**.

Step-by-Step Procedure

1. Begin with graph G .
2. Randomly select an edge.
3. Contract the selected edge.
4. Continue contracting edges until only two vertices remain.
5. The edges connecting those vertices represent the resulting cut.

The diagram on **page 34** demonstrates an example execution of Karger's algorithm.

12. Recursive Randomized Min-Cut Pseudocode

Algorithm: Recursive-Randomized-Min-Cut(G, α)

Input:

- Undirected multigraph G with n vertices
- Constant $\alpha > 0$

Output:

- Cut C
-

Algorithm Steps

1. If

$$n \leq \alpha^3$$

then compute the minimum cut using **brute force search**.

2. Otherwise:

Repeat for $i = 1$ to α :

a. Create graph G' by applying random contraction steps.

b. Recursively compute

$$C' = \text{RecursiveRandomizedMinCut}(G', \alpha)$$

c. If

$$i = 1 \text{ or } |C'| < |C|$$

then update

$$C = C'$$

3. Return the cut C .

13. Comparison of Deterministic and Randomized Min-Cut

Deterministic Algorithm

Characteristics:

- Always returns the **exact minimum cut**.
- May require higher computational time.

Time complexity:

$$O(VE + V^2 \log V)$$

Randomized Algorithm

Characteristics:

- Produces an **approximate result with high probability**.
- Faster for large graphs.

Time complexity:

$$O(V^2)$$

Therefore, the choice between deterministic and randomized algorithms depends on the **specific problem requirements**.

14. Probability Theorem for Min-Cut

The lecture provides a probability guarantee for the randomized algorithm.

The probability that the algorithm outputs the correct minimum cut is:

$$\frac{2}{n(n-1)}$$

where n is the number of vertices.

Because the probability of success in one run is small, the algorithm is typically **repeated multiple times** to increase the likelihood of finding the true minimum cut.

15. Python Simulation

The lecture concludes with instructions for simulation:

Students should:

1. Open the **Campuswire post related to Lecture 10**.
 2. Download the **Jupyter Notebook (.ipynb) file** provided in the comments.
 3. Run the simulation to observe the randomized algorithm in action.
-

Final Summary

This lecture explains the **minimum cut problem in graphs** and introduces both deterministic and randomized methods to solve it.

Key concepts include:

- Cut-sets and minimum cuts
- Edge contraction operation
- Successful and unsuccessful runs of randomized algorithms

- Max-flow min-cut theorem
- Stoer–Wagner deterministic algorithm
- Karger’s randomized min-cut algorithm
- Time complexity and probability guarantees

These techniques illustrate how **randomization can simplify complex graph problems while maintaining high accuracy.**

✅ If you want, I can also make a **super exam-focused version of this lecture:**

- 10 long answers (6–8 marks each)
- 15 short notes (2–3 marks)
- Predicted 40-mark exam paper
- 1-page last-minute cheat sheet

This usually helps students **score very high in theory exams.**

Generate a lecture scribe based on the attached materials that is intended to serve as reference material for exam preparation. While writing the scribe, you must reason step-by-step through all definitions, assumptions, proofs, and example solutions exactly as they are presented in the provided context.

Ensure the logical flow is made explicit and all reasoning is included in the final scribe so a student can follow it during revision. Do not invent content, introduce new examples, or include information not found in the lecture slides or textbook. Maintain a formal academic tone and follow the structure and scope of the lecture.

Below is a **formal lecture scribe for exam preparation**, constructed strictly from the uploaded lecture slides. The content follows the **same structure, terminology, and logical order as the lecture material**. All explanations reconstruct the reasoning presented in the slides so that a student can **follow the concepts step-by-step during revision**. No external information or additional examples have been introduced.

Source:

Lecture Scribe

CSE400 – Fundamentals of Probability in Computing

Lecture 10: Randomized Min-Cut Algorithm

1. Introduction

This lecture studies the **Minimum Cut problem in graphs** and explains how it can be solved using **deterministic algorithms and randomized algorithms**. The lecture gradually introduces the concept of a cut, the definition of the minimum cut, and the algorithmic approaches used to compute it.

The lecture structure includes the following topics:

1. The Min-Cut Problem
2. Why Min-Cut is useful
3. Definition of Min-Cut
4. Successful Min-Cut Run
5. Unsuccessful Min-Cut Run
6. Max-Flow Min-Cut Theorem
7. Deterministic Min-Cut Algorithm
8. Stoer–Wagner Minimum Cut Algorithm
9. Pseudocode of the deterministic algorithm
10. Randomized Min-Cut Algorithm
11. Karger's Randomized Algorithm
12. Pseudocode for the randomized algorithm
13. Comparison between deterministic and randomized approaches
14. Probability theorem for min-cut set
15. Python simulation instructions

These topics collectively explain how **graph partitioning and network reliability problems** can be solved algorithmically.

2. The Min-Cut Problem

2.1 Motivation: Why Use the Min-Cut Algorithm

The min-cut algorithm is applied in various scenarios where **network connectivity, reliability, and optimization** must be analyzed.

The lecture identifies three major application areas.

2.1.1 Network Design

In network design, the goal is to ensure efficient communication and optimal data flow.

Step-by-step reasoning:

1. A communication network can be modeled as a **graph**.
2. The nodes represent devices or communication endpoints.
3. The edges represent communication links.
4. Some edges may be more critical than others.
5. If certain edges are removed, the network may break into separate parts.

The min-cut algorithm identifies the **minimum capacity cut**, which corresponds to the smallest set of edges whose removal disconnects the network.

Therefore, network designers use the min-cut algorithm to **improve communication efficiency and optimize network flow**.

2.1.2 Communication Networks

Communication networks must remain operational even when failures occur.

Logical reasoning:

1. Networks may experience failures such as broken links or overloaded channels.
2. Some links are critical for maintaining connectivity.
3. If only a few edges can disconnect the network, the system is vulnerable.

The min-cut algorithm identifies these critical edges. By recognizing them, engineers can design **fault-tolerant communication networks**.

Thus, the algorithm helps understand the **vulnerability of communication networks to failures**.

2.1.3 VLSI Design

Very Large Scale Integration (VLSI) involves designing circuits with extremely large numbers of components.

Reasoning:

1. Circuit components can be represented as vertices.
2. Connections between components are represented as edges.
3. Very large circuits often need to be partitioned into smaller sections.
4. Partitioning should minimize the number of connections between different sections.

The min-cut algorithm assists in partitioning circuits so that **interconnection complexity is reduced**.

3. Definition of Min-Cut

3.1 Cut-Set

A **cut-set** in a graph is defined as:

A set of edges whose removal breaks the graph into two or more connected components.

Logical Explanation

To understand this definition step-by-step:

1. A graph consists of vertices connected by edges.
2. These edges maintain connectivity among the vertices.
3. If certain edges are removed, connectivity may be lost.
4. When the removal of specific edges divides the graph into separate connected components, those edges form a **cut-set**.

Thus, a cut-set represents **edges whose removal disconnects the graph**.

3.2 Minimum Cut

Consider a graph

$$G = (V, E)$$

where

- V is the set of vertices
- E is the set of edges

- n represents the number of vertices

The **minimum cut problem** is defined as:

Finding a cut-set of minimum cardinality in graph G .

Step-by-Step Reasoning

1. A graph may have multiple cut-sets.
2. Each cut-set has a certain number of edges.
3. Among all these cut-sets, the smallest one is the **minimum cut**.

Therefore,

Minimum Cut = cut-set with the smallest number of edges

4. Edge Contraction

The lecture identifies **edge contraction** as the main operation used in the min-cut algorithm.

Definition

Edge contraction is defined as:

- Removing an edge from the graph
- Merging the two vertices connected by that edge into a single vertex

This process is illustrated in the diagram on **page 15 of the slides**, where two vertices connected by an edge become a single merged vertex after contraction.

Detailed Contraction Procedure

When contracting an edge (u, v) :

1. Select the edge connecting vertices u and v .
2. Merge vertices u and v into a new single vertex.
3. Remove the edge connecting them.
4. Preserve all other edges in the graph.

Two additional properties must be considered:

- The new graph may contain **parallel edges**.
- **Self-loops are removed**.

Thus the contraction operation simplifies the graph while maintaining its overall connectivity structure.

5. Successful Min-Cut Run

A **successful min-cut run** occurs when the algorithm correctly identifies the minimum cut.

Logical Reasoning

1. The algorithm repeatedly performs edge contractions.
2. Some edges belong to the true minimum cut.
3. If the algorithm **does not contract those edges**, the minimum cut remains preserved.
4. The graph is gradually reduced until only two vertices remain.
5. The edges between these two vertices represent the minimum cut.

The diagram on **page 19** demonstrates such a successful contraction sequence.

6. Unsuccessful Min-Cut Run

An **unsuccessful run** occurs when the algorithm fails to produce the correct minimum cut.

Logical Reasoning

1. The algorithm randomly selects edges for contraction.
2. If an edge belonging to the true minimum cut is contracted early,
3. the structure of the minimum cut is destroyed.
4. The final cut produced by the algorithm will therefore be incorrect.

The example on **page 22** illustrates such an unsuccessful run.

7. Max-Flow Min-Cut Theorem

The lecture introduces the **Max-Flow Min-Cut Theorem**, which establishes a fundamental relationship between network flow and graph cuts.

Theorem Statement

In a flow network, the maximum amount of flow passing from the source to the sink is equal to the total weight of the edges in a minimum cut.

Definitions Used in the Theorem

Capacity of a Cut

The capacity of a cut is defined as the **sum of the capacities of edges that go from set X to set Y** .

Minimum Cut

The cut with the **smallest possible capacity**.

Minimum Cut Capacity

The total capacity of the edges that belong to the minimum cut.

Maximum Flow

The largest possible flow that can be sent from:

- Source S
- to Sink T

in the network.

The network diagram shown on **page 26** illustrates this relationship between flow and cut capacity.

8. Deterministic Min-Cut Algorithm

Deterministic algorithms guarantee an exact solution.

The lecture introduces the **Stoer–Wagner Minimum Cut Algorithm** as a deterministic approach.

Stoer–Wagner Theorem

Let

- s and t be vertices in graph G
- $G/s, t$ represent the graph formed by merging vertices s and t

The theorem states that a minimum cut of G can be obtained by selecting the smaller value between:

1. A minimum $s - t$ cut of G
2. A minimum cut of the merged graph $G/s, t$

Logical Explanation

Two possibilities exist:

Case 1

The minimum cut separates s and t .

In this situation, the minimum $s - t$ cut of G is also the minimum cut of the graph.

Case 2

The minimum cut does not separate s and t .

In this case merging them does not affect the minimum cut, so the minimum cut of $G/s, t$ becomes the correct result.

9. Deterministic Algorithm Pseudocode

Algorithm 1: MinimumCutPhase(G, a)

Step-by-step procedure:

1. Initialize the set

$$A = \{a\}$$

2. While

$$A \neq V$$

add the vertex that is most tightly connected to set A .

3. After all vertices are included in A , return the **cut weight**, which is called the **cut of the phase**.

Algorithm 2: MinimumCut(G)

1. While the graph contains vertices:
2. Choose any vertex a .
3. Execute

$$\text{MinimumCutPhase}(G, a)$$

4. Compare the cut obtained with the current minimum cut.
5. If the phase cut is smaller, update the stored minimum cut.
6. Shrink the graph by merging the last two vertices added.
7. Continue the process until all phases are completed.

Finally, return the **minimum cut**.

10. Randomized Min-Cut Algorithm

Randomized algorithms use random decisions during execution.

Why Randomized Algorithms Are Used

The lecture lists several advantages:

- Probabilistic guarantee of success
- Efficiency
- Parallelization
- Approximation guarantees
- Avoidance of worst-case instances
- Robustness

These properties make randomized algorithms useful when deterministic algorithms become computationally expensive.

11. Karger's Randomized Min-Cut Algorithm

Karger's algorithm finds the minimum cut using **random edge contractions**.

Algorithm Procedure

1. Start with graph G .
2. Randomly select an edge.
3. Contract the selected edge.
4. Continue performing random contractions.
5. Stop when only two vertices remain.

The edges connecting the remaining two vertices form the resulting cut.

The diagram on **page 34** illustrates an example execution of Karger's algorithm.

12. Recursive Randomized Min-Cut Pseudocode

Algorithm: Recursive-Randomized-Min-Cut(G, α)

Input:

- Undirected multigraph G
- Constant $\alpha > 0$

Output:

- A cut C

Algorithm Steps

1. If

$$n \leq \alpha^3$$

then compute the minimum cut using **brute force search**.

2. Otherwise repeat the following for $i = 1$ to α :

- Generate graph G' by performing random contraction steps.
- Compute recursively

$$C' = \text{RecursiveRandomizedMinCut}(G', \alpha)$$

c. If

$$i = 1 \text{ or } |C'| < |C|$$

update

$$C = C'$$

3. Return C .

13. Comparison of Deterministic and Randomized Algorithms

Deterministic Min-Cut

Characteristics:

- Always produces the exact minimum cut.
- May require higher computation time.

Time complexity:

$$O(VE + V^2 \log V)$$

Randomized Min-Cut

Characteristics:

- Produces an approximate minimum cut with high probability.
- Faster for large graphs.

Time complexity:

$$O(V^2)$$

The selection between these approaches depends on the **specific problem and computational constraints**.

14. Probability Theorem for Min-Cut

The lecture states a probability bound for the randomized algorithm.

The probability that the algorithm outputs the correct minimum cut is at least

$$\frac{2}{n(n-1)}$$

where n is the number of vertices.

Because this probability may be small for large graphs, the algorithm is typically **repeated multiple times** to increase the chance of success.

15. Python Simulation

Students are instructed to:

1. Open the **Campuswire post related to Lecture 10**.
2. Download the **Jupyter Notebook (.ipynb)** file provided in the comments.
3. Run the notebook to simulate the randomized min-cut algorithm.

Final Summary

This lecture introduces the **minimum cut problem and algorithms used to solve it**.

The key ideas developed throughout the lecture include:

- Definition of cut-sets and minimum cuts
- Edge contraction as the fundamental operation
- Successful and unsuccessful runs of randomized algorithms
- Max-flow min-cut theorem
- Stoer–Wagner deterministic algorithm
- Karger's randomized min-cut algorithm
- Probability guarantees for randomized algorithms

These concepts demonstrate how **graph theory and probabilistic algorithms can be used to analyze and optimize network structures**.

If you'd like, I can also create a **super exam-focused pack for this lecture** including:

- 10 ready-to-write long answers (6–8 marks)
- 2–3 mark short notes
- Predicted 40-mark theory paper
- 1-page revision cheat sheet

This is usually the **fastest way to prepare the night before the exam**.

Generate a lecture scribe based on the attached materials that is intended to serve as reference material for exam preparation. While writing the scribe, you must reason step-by-step through all definitions, assumptions, proofs, and example solutions exactly as they are presented in the provided context.

Ensure the logical flow is made explicit and all reasoning is included in the final scribe so a student can follow it during revision. Do not invent content, introduce new

examples, or include information not found in the lecture slides or textbook. Maintain a formal academic tone and follow the structure and scope of the lecture.

Below is a **formal lecture scribe** constructed strictly from the provided lecture slides for **CSE400 – Fundamentals of Probability in Computing, Lecture 10: Randomized Min-Cut Algorithm**. The explanations follow the **same structure, sequence, and terminology** as the slides, and the reasoning is reconstructed step-by-step so that the material can serve as **reference notes for exam preparation**. No additional examples or external information have been introduced.

Source:

Lecture Scribe

CSE400 – Fundamentals of Probability in Computing

Lecture 10: Randomized Min-Cut Algorithm

1. Lecture Overview

This lecture introduces the **minimum cut problem in graphs** and studies both **deterministic and randomized algorithms** used to solve it. The topics covered in the lecture include:

1. The Min-Cut Problem
2. Why the Min-Cut algorithm is useful
3. Definition of Min-Cut
4. Successful Min-Cut Run
5. Unsuccessful Min-Cut Run
6. Max-Flow Min-Cut Theorem
7. Deterministic Min-Cut Algorithm
8. Stoer–Wagner Minimum Cut Algorithm
9. Pseudocode for deterministic min-cut
10. Randomized Min-Cut Algorithm
11. Karger's Randomized Algorithm
12. Pseudocode of the randomized algorithm
13. Comparison of deterministic and randomized min-cut algorithms
14. Theorem describing the probability of finding a min-cut
15. Python simulation instructions

The lecture aims to show how **graph partitioning and network reliability problems** can be solved using algorithmic techniques.

2. The Min-Cut Problem

2.1 Why Use Min-Cut?

The min-cut algorithm is applied in problems related to **network connectivity, reliability, and optimization**.

The lecture describes several application areas.

2.1.1 Network Design

Min-cut helps improve communication efficiency and optimize network flow.

Step-by-step reasoning:

1. A network can be represented using a graph.
2. Nodes represent devices or entities in the network.
3. Edges represent communication links.
4. Removing certain links may break the network connectivity.
5. The goal is to determine the **minimum capacity cut**, which is the smallest set of edges whose removal disconnects the network.

By identifying this set, engineers can analyze weaknesses and optimize the network design.

2.1.2 Communication Networks

Min-cut can help understand network vulnerability.

Reasoning process:

1. Communication networks must remain functional even when some components fail.
2. If a small number of edges can disconnect the network, the network is vulnerable.
3. By computing the minimum cut, these critical edges can be identified.

This helps design **robust and fault-tolerant communication systems**.

2.1.3 VLSI Design

In Very Large Scale Integration (VLSI) systems, circuits contain many interconnected components.

Reasoning:

1. Circuit components can be modeled as vertices.
2. Interconnections between components are represented as edges.
3. Large circuits often need to be partitioned into smaller components.
4. Partitioning should minimize the number of connections between partitions.

The min-cut algorithm helps perform such partitioning efficiently and reduces interconnect complexity.

3. Definition of Min-Cut

3.1 Cut-Set

A **cut-set** in a graph is defined as:

A set of edges whose removal breaks the graph into two or more connected components.

Step-by-Step Interpretation

1. A graph consists of vertices connected by edges.
2. The edges maintain connectivity between vertices.
3. If certain edges are removed, connectivity may be disrupted.
4. When removing a set of edges divides the graph into separate components, those edges form a **cut-set**.

Thus, a cut-set represents **edges responsible for maintaining connectivity between different parts of the graph**.

3.2 Minimum Cut

Given a graph

$$G = (V, E)$$

where

- V is the set of vertices

- E is the set of edges
- n is the number of vertices

The **minimum cut problem** is defined as:

Finding a cut-set with the smallest possible number of edges in graph G .

Logical Reasoning

1. A graph may contain several different cut-sets.
 2. Each cut-set contains a specific number of edges.
 3. The cut-set with the smallest number of edges is called the **minimum cut**.
-

4. Edge Contraction

The lecture identifies **edge contraction** as the main operation used in the min-cut algorithm.

Definition

Edge contraction is an operation that:

- Removes an edge from a graph
- Merges the two vertices connected by that edge into a single vertex

This process is illustrated in the diagram shown on **page 15 of the slides**, where two vertices connected by an edge are combined into one vertex after contraction.

Detailed Edge Contraction Procedure

When contracting an edge (u, v) :

1. Select the edge connecting vertices u and v .
2. Merge vertices u and v into a single vertex.
3. Remove the edge connecting them.
4. Retain all other edges in the graph.

Two additional properties are mentioned:

- The resulting graph may contain **parallel edges**.
- **Self-loops are removed** from the graph.

Thus, the contraction operation simplifies the graph while maintaining the essential connectivity relationships.

5. Successful Min-Cut Run

A **successful min-cut run** refers to a case where the algorithm successfully finds the minimum cut.

Step-by-Step Reasoning

1. The algorithm repeatedly performs edge contractions.
2. Some edges belong to the true minimum cut.
3. If these edges are **not contracted**, the structure of the minimum cut remains intact.
4. The graph eventually reduces to two vertices.
5. The edges connecting these vertices represent the minimum cut.

The example shown on **page 19 of the slides** illustrates a sequence of contractions that leads to a correct minimum cut.

6. Unsuccessful Min-Cut Run

An **unsuccessful run** occurs when the algorithm fails to find the correct minimum cut.

Logical Reasoning

1. The algorithm selects edges randomly for contraction.
2. If an edge belonging to the true minimum cut is contracted early,
3. the structure of the minimum cut is destroyed.
4. The final cut produced by the algorithm will therefore not be the true minimum cut.

The example on **page 22 of the slides** illustrates this situation.

7. Max-Flow Min-Cut Theorem

The **Max-Flow Min-Cut Theorem** establishes a relationship between network flow and graph cuts.

Theorem Statement

In a flow network, the maximum amount of flow passing from the source to the sink is equal to the total weight of the edges in a minimum cut.

Definitions Used in the Theorem

Capacity of a Cut

The capacity of a cut is defined as:

The sum of the capacities of edges oriented from a vertex in set X to a vertex in set Y .

Minimum Cut

The cut that has the **smallest possible capacity**.

Minimum Cut Capacity

The capacity of the minimum cut.

Maximum Flow

The largest amount of flow that can travel from:

- Source S
- to Sink T

in the network.

The network diagram shown on **page 26** illustrates this relationship between maximum flow and minimum cut.

8. Deterministic Min-Cut Algorithm

A deterministic algorithm guarantees the **exact minimum cut**.

The lecture introduces the **Stoer–Wagner Minimum Cut Algorithm**.

Stoer–Wagner Theorem

Let

- s and t be two vertices in graph G
- $G/s, t$ represent the graph obtained by merging vertices s and t

The theorem states:

A minimum cut of G can be obtained by taking the smaller value between:

1. A minimum $s - t$ cut of G
2. A minimum cut of the graph $G/s, t$

Logical Explanation

Two possibilities exist:

Case 1

There exists a minimum cut that separates vertices s and t .

In this case, the **minimum $s - t$ cut** is also the minimum cut of the graph.

Case 2

There is no minimum cut that separates s and t .

In this situation, merging s and t does not affect the minimum cut, so the minimum cut of $G/s, t$ becomes the solution.

9. Deterministic Min-Cut Algorithm Pseudocode

Algorithm 1: MinimumCutPhase(G, a)

1. Initialize the set

$$A = \{a\}$$

2. While

$$A \neq V$$

add the vertex that is most tightly connected to set A .

3. Once all vertices are included in A , return the **cut weight**, called the **cut of the phase**.

Algorithm 2: MinimumCut(G)

1. While the graph contains vertices:
2. Choose any vertex a .
3. Execute the procedure

$$\text{MinimumCutPhase}(G, a)$$

4. If the cut obtained in the phase is smaller than the current minimum cut, update the minimum cut.
 5. Shrink the graph by merging the last two vertices added.
 6. Continue until the algorithm terminates.
 7. Return the minimum cut.
-

10. Randomized Min-Cut Algorithm

Randomized algorithms incorporate **random decisions** during execution.

Why Randomized Algorithms Are Used

The lecture lists several reasons:

- Probabilistic guarantee of success
- Efficiency
- Parallelization
- Approximation guarantees
- Avoidance of worst-case instances
- Robustness

These properties make randomized algorithms useful when deterministic algorithms become computationally expensive.

11. Karger's Randomized Algorithm

Karger's algorithm finds the minimum cut through **random edge contractions**.

Algorithm Steps

1. Start with graph G .
2. Randomly choose an edge.
3. Contract the selected edge.
4. Repeat the process until only two vertices remain.
5. The edges connecting the two vertices form the resulting cut.

The diagram shown on **page 34 of the slides** illustrates an example run of Karger's algorithm.

12. Recursive Randomized Min-Cut Pseudocode

Algorithm: Recursive-Randomized-Min-Cut(G, α)

Input:

- Undirected multigraph G with n vertices
- Integer constant $\alpha > 0$

Output:

- A cut C

Algorithm Steps

1. If

$$n \leq \alpha^3$$

then compute the minimum cut using **brute force search**.

2. Otherwise repeat for $i = 1$ to α :

- Generate graph G' by performing random contraction steps.
- Compute recursively

$$C' = \text{RecursiveRandomizedMinCut}(G', \alpha)$$

c. If

$$i = 1 \text{ or } |C'| < |C|$$

update

$$C = C'$$

3. Return C .

13. Comparison: Deterministic vs Randomized Min-Cut

Deterministic Min-Cut

Characteristics:

- Always guarantees an exact minimum cut.
- May have higher time complexity for large graphs.

Time complexity:

$$O(VE + V^2 \log V)$$

Randomized Min-Cut

Characteristics:

- Produces an approximate minimum cut with high probability.
- Faster for large graphs.

Time complexity:

$$O(V^2)$$

Thus, the choice of algorithm depends on the **specific problem requirements**.

14. Probability Theorem for Min-Cut

The lecture provides the probability guarantee for the randomized algorithm.

The probability that the algorithm outputs a minimum cut is at least

$$\frac{2}{n(n-1)}$$

where n is the number of vertices.

Because this probability can be small, the algorithm is typically **executed multiple times** to increase the likelihood of finding the correct minimum cut.

15. Python Simulation

Students are instructed to:

1. Open the **Campuswire post for Lecture 10**.
 2. Download the **Jupyter Notebook (.ipynb)** file provided in the comments.
 3. Execute the notebook to observe the randomized min-cut algorithm in practice.
-

Final Summary

This lecture introduces the **minimum cut problem** and the **algorithms used to solve it**.

The main ideas developed throughout the lecture include:

- Definition of cut-sets and minimum cuts
- Edge contraction as the key operation in min-cut algorithms
- Successful and unsuccessful runs of randomized algorithms
- The Max-Flow Min-Cut theorem
- The Stoer–Wagner deterministic algorithm
- Karger’s randomized min-cut algorithm
- Complexity analysis and probability guarantees

Together, these concepts illustrate how **graph algorithms and probabilistic techniques can be applied to analyze and optimize networks.**

If you want, I can also make a **very short 2-page “night-before-exam” version of this lecture** (only the most important definitions, algorithms, and formulas). It helps a lot when revising quickly before the exam.